

CSC311: Data Structures

Algorithm Efficiency

Dr. Amlan Chatterjee

Computer Science Department
California State University, Dominguez Hills

What is an Algorithm?

- An algorithm is a set of step-by-step operations to perform a specific task.
- The characteristics of an algorithm are:
 - Precision – the steps are precisely defined and unambiguous.
 - Uniqueness – results of each step are uniquely defined and only depend on the input and the result of the preceding steps.
 - Finiteness – the algorithm stops after a finite number of instructions are executed.
 - Input – the algorithm receives input.
 - Output – the algorithm produces output.

Algorithm Efficiency and Big-O

- Getting a precise measure of the performance of an algorithm is difficult
- Big-O notation expresses the performance of an algorithm as a function of the number of items to be processed
- This permits algorithms to be compared for efficiency
- For more than a certain number of data items, some problems cannot be solved by any computer

Problem 1: Searching for a value

- Given an array of integer values, search if a specific item is present.

Problem 1: Searching for a value

- Given an array of integer values, search if a specific item is present.

```
public static int search(int[] x, int target) {  
    for(int i=0; i < x.length; i++) {  
        if (x[i]==target)  
            return i;  
    }  
    return -1; // target not found  
}
```

Linear Growth Rate

If processing time increases in proportion to the number of inputs n , the algorithm grows at a linear rate

This search technique is also known as *linear search*

```
public static int search(int[] x, int target) {  
    for(int i=0; i < x.length; i++) {  
        if (x[i]==target)  
            return i;  
    }  
    return -1; // target not found  
}
```

Linear Growth Rate

If processing time is a function of inputs n , the algorithm has a linear growth rate. This search technique is an example of linear growth rate.

- If the target is not present, the for loop will execute `x.length` times
- If the target is present the for loop will execute (on average) $(x.length + 1)/2$ times
- Therefore, the total execution time is directly proportional to `x.length`
- This is described as a growth rate of order n or $O(n)$

```
public static int search(int[] x, int target) {  
    for(int i=0; i < x.length; i++) {  
        if (x[i]==target)  
            return i;  
    }  
    return -1; // target not found  
}
```

Problem 2: Comparing items in 2 different arrays

- Given 2 arrays of integer values, check if all the values of one is present in the other

Problem 2: Comparing items in 2 different arrays

- Given 2 arrays of integer values, check if all the values of one is present in the other

```
public static boolean areDifferent(int[] x, int[] y)
{
    for(int i=0; i < x.length; i++) {
        if (search(y, x[i]) != -1)
            return false;
    }
    return true;
}
```

n x m Growth Rate

Processing time can be dependent on two different inputs

```
public static boolean areDifferent(int[] x, int[] y) {  
    for(int i=0; i < x.length; i++) {  
        if (search(y, x[i]) != -1)  
            return false;  
    }  
    return true;  
}
```

n x m Growth Rate (cont.)

Processing time
different inputs.

- The for loop will execute `x.length` times
- But it will call `search`, which will execute `y.length` times
- The total execution time is proportional to $(x.length * y.length)$
- The growth rate has an order of $n \times m$ or $O(n \times m)$

```
public static boolean areDifferent(int[] x, int[] y) {  
    for(int i=0; i < x.length; i++) {  
        if (search(y, x[i]) != -1)  
            return false;  
    }  
    return true;  
}
```

Problem 3: Check if values in an array are unique

- Given an array of integer values, check to see if all are unique

Problem 3: Check if values in an array are unique

- Given an array of integer values, check to see if all are unique

```
public static boolean areUnique(int[] x) {  
    for(int i=0; i < x.length; i++) {  
        for(int j=0; j < x.length; j++) {  
            if (i != j && x[i] == x[j])  
                return false;  
        }  
    }  
    return true;  
}
```

Quadratic Growth Rate

If processing time is proportional to the square of the number of inputs n , the algorithm grows at a quadratic rate

```
public static boolean areUnique(int[] x) {  
    for(int i=0; i < x.length; i++) {  
        for(int j=0; j < x.length; j++) {  
            if (i != j && x[i] == x[j])  
                return false;  
        }  
    }  
    return true;  
}
```

Quadratic Growth Rate (cont.)

If processing time
the number of inputs is
quadratic rate

- The for loop with `i` as index will execute `x.length` times
- The for loop with `j` as index will execute `x.length` times
- The total number of times the inner loop will execute is $(x.length)^2$
- The growth rate has an order of n^2 or $O(n^2)$

```
public static boolean areUnique(int[] x) {  
    for(int i=0; i < x.length; i++) {  
        for(int j=0; j < x.length; j++) {  
            if (i != j && x[i] == x[j])  
                return false;  
        }  
    }  
    return true;  
}
```

Why efficient code?

- Computers are faster, have larger memories
 - So why worry about efficient code?
- And ... how do we measure efficiency?

Example

Consider the problem of summing

$$\sum_{k=1}^n k = 1 + 2 + 3 + \dots + n$$

Three algorithms for computing the sum
 $1 + 2 + \dots + n$, for an integer $n > 0$

Algorithm A	Algorithm B	Algorithm C
<pre>sum = 0 for i = 1 to n sum = sum + i</pre>	<pre>sum = 0 for i = 1 to n { for j = 1 to i sum = sum + 1 }</pre>	<pre>sum = n * (n + 1) / 2</pre>

Example: Java code for the 3 algorithms

```
// Computing the sum of the consecutive integers from 1 to n:
long n = 10000; // Ten thousand

// Algorithm A
long sum = 0;
for (long i = 1; i <= n; i++)
    sum = sum + i;
System.out.println(sum);

// Algorithm B
sum = 0;
for (long i = 1; i <= n; i++)
{
    for (long j = 1; j <= i; j++)
        sum = sum + 1;
} // end for
System.out.println(sum);

// Algorithm C
sum = n * (n + 1) / 2;
System.out.println(sum);
```

What is “best”?

- An algorithm has both time and space constraints – that is complexity
 - Time complexity
 - Space complexity
- This study is called analysis of algorithms
- Right now we are interested in studying the time complexity only

Big-O Notation

- The $O()$ in the previous examples can be thought of as an abbreviation of "*order of magnitude*"
- A simple way to determine the Big-O notation of an algorithm is to look at the loops and to see whether the loops are nested
- Assuming a loop body consists only of simple statements,
 - a single loop is $O(n)$
 - a pair of nested loops is $O(n^2)$
 - a nested pair of loops inside another is $O(n^3)$
 - and so on . . .

Big-O Notation (cont.)

- You must also examine the *number of times* a loop is executed

```
for(i=1; i < x.length; i *= 2) {  
    // Do something with x[i]  
}
```
- The loop body will execute $k-1$ times, with i having the following values:
1, 2, 4, 8, 16, ..., 2^k
until 2^k is greater than $x.length$
- Since $2^{k-1} = x.length < 2^k$ and $\log_2 2^k$ is k , we know that $k-1 = \log_2(x.length) < k$
- Thus we say the loop is $O(\log n)$ (in analyzing algorithms, we use logarithms to the base 2)
- Logarithmic functions grow slowly as the number of data items n increases

Formal Definition of Big-O

Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}  
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}  
Simple Statement 6  
Simple Statement 7  
...  
Simple Statement 30
```

Formal Definition of Big-O (cont.)

Consider the following program structure:

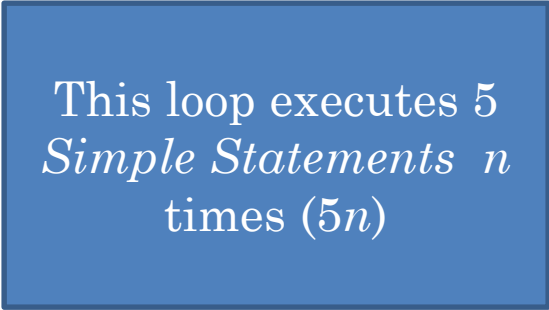
```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}  
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}  
Simple Statement 6  
Simple Statement 7  
...  
Simple Statement 30
```

This nested loop
executes a *Simple
Statement* n^2 times

Formal Definition of Big-O (cont.)

Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}  
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}  
Simple Statement 6  
Simple Statement 7  
...  
Simple Statement 30
```

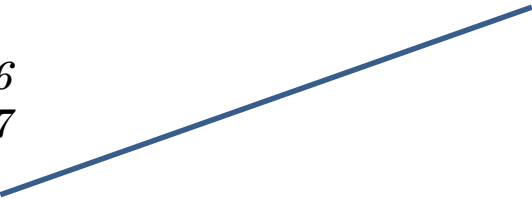


This loop executes 5
Simple Statements n
times ($5n$)

Formal Definition of Big-O (cont.)

Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}  
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}  
Simple Statement 6  
Simple Statement 7  
...  
Simple Statement 30
```



Finally, 25 *Simple Statements* are executed

Formal Definition of Big-O (cont.)

Consider the following program structure:

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        Simple Statement  
    }  
}  
for (int i = 0; i < n; i++) {  
    Simple Statement 1  
    Simple Statement 2  
    Simple Statement 3  
    Simple Statement 4  
    Simple Statement 5  
}  
Simple Statement 6  
Simple Statement 7  
...  
Simple Statement 30
```

We can conclude that the relationship between processing time and n (the number of data items processed) is:

$$T(n) = n^2 + 5n + 25$$

Formal Definition of Big-O (cont.)

- In terms of $T(n)$,
 - $T(n) = O(f(n))$

if there exists

- two constants, n_0 and c , greater than zero, and
- a function, $f(n)$,
such that for all $n > n_0$, $cf(n) > T(n)$

- In other words, as n gets sufficiently large (larger than n_0), there is some constant c for which the processing time will always be less than or equal to $cf(n)$
- $cf(n)$ is an upper bound on performance

Formal Definition of Big-O (cont.)

The following is a precise definition for an upper bound. $\mathbf{T}(n)$ represents the true running time of the algorithm. $f(n)$ is some expression for the upper bound.

For $\mathbf{T}(n)$ a non-negatively valued function, $\mathbf{T}(n)$ is in set $O(f(n))$ if there exist two positive constants c and n_0 such that $\mathbf{T}(n) \leq cf(n)$ for all $n > n_0$.

Knowing that something is in $O(f(n))$ says only how bad things can be.

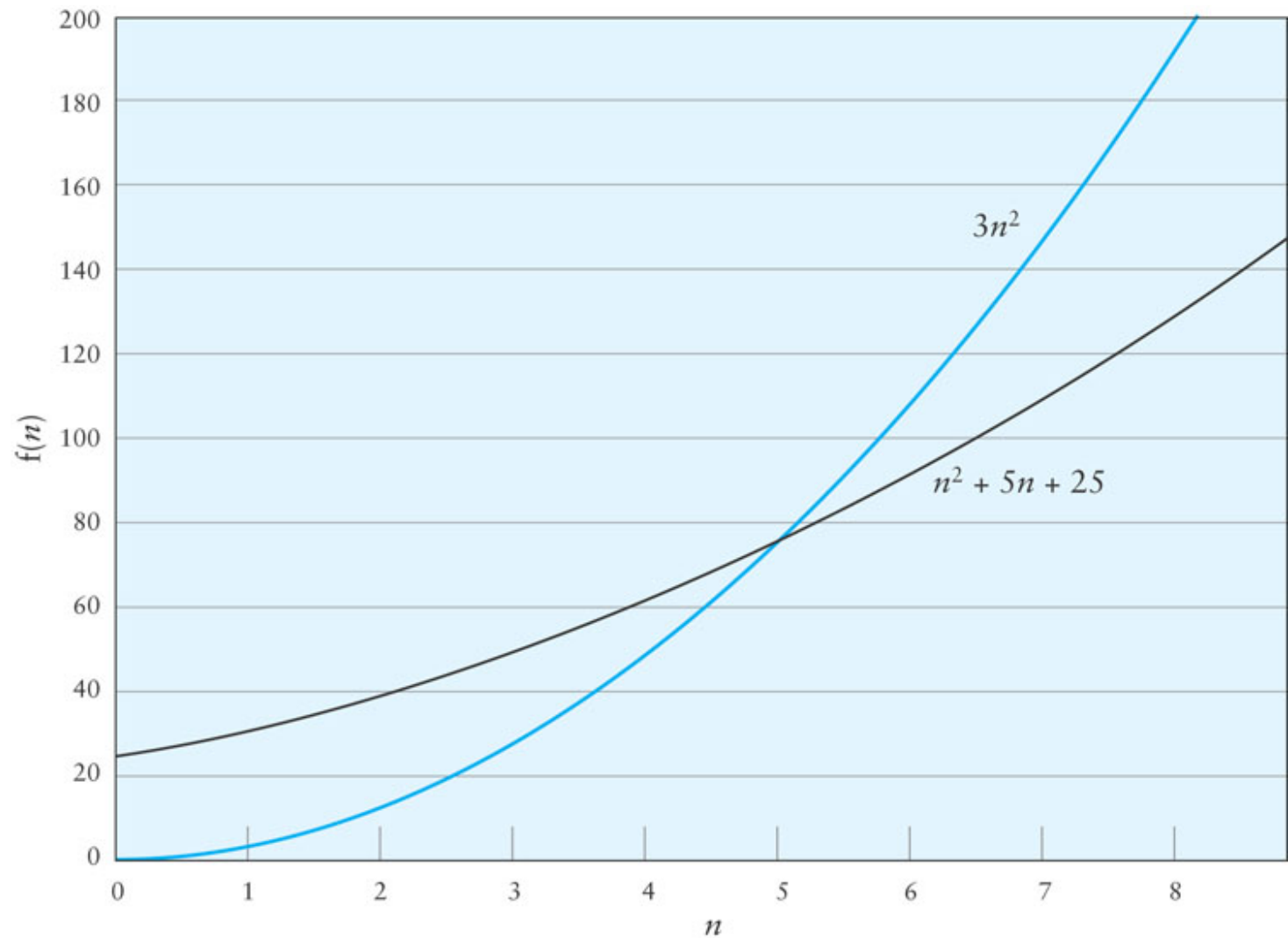
Formal Definition of Big-O (cont.)

- The growth rate of $f(n)$ will be determined by the fastest growing term, which is the one with the largest exponent
- In the example, an algorithm of
 $O(n^2 + 5n + 25)$
is more simply expressed as
 $O(n^2)$
- In general, it is safe to ignore all constants and to drop the lower-order terms when determining the order of magnitude

Big-O Example 1

- Given $T(n) = n^2 + 5n + 25$, show that this is $O(n^2)$
- Find constants n_0 and c so that, for all $n > n_0$,
 $cn^2 > n^2 + 5n + 25$
 - Find the point where $cn^2 = n^2 + 5n + 25$
 - Let $n = n_0$, and solve for c
 $c = 1 + 5/n_0 + 25/n_0^2$
- When n_0 is 5
 $c = (1 + 5/5 + 25/25)$, i.e., c is 3
- So, $3n^2 > n^2 + 5n + 25$ for all $n > 5$
- Other values of n_0 and c also work

Big-O Example 1 (cont.)



Big-O Example 2

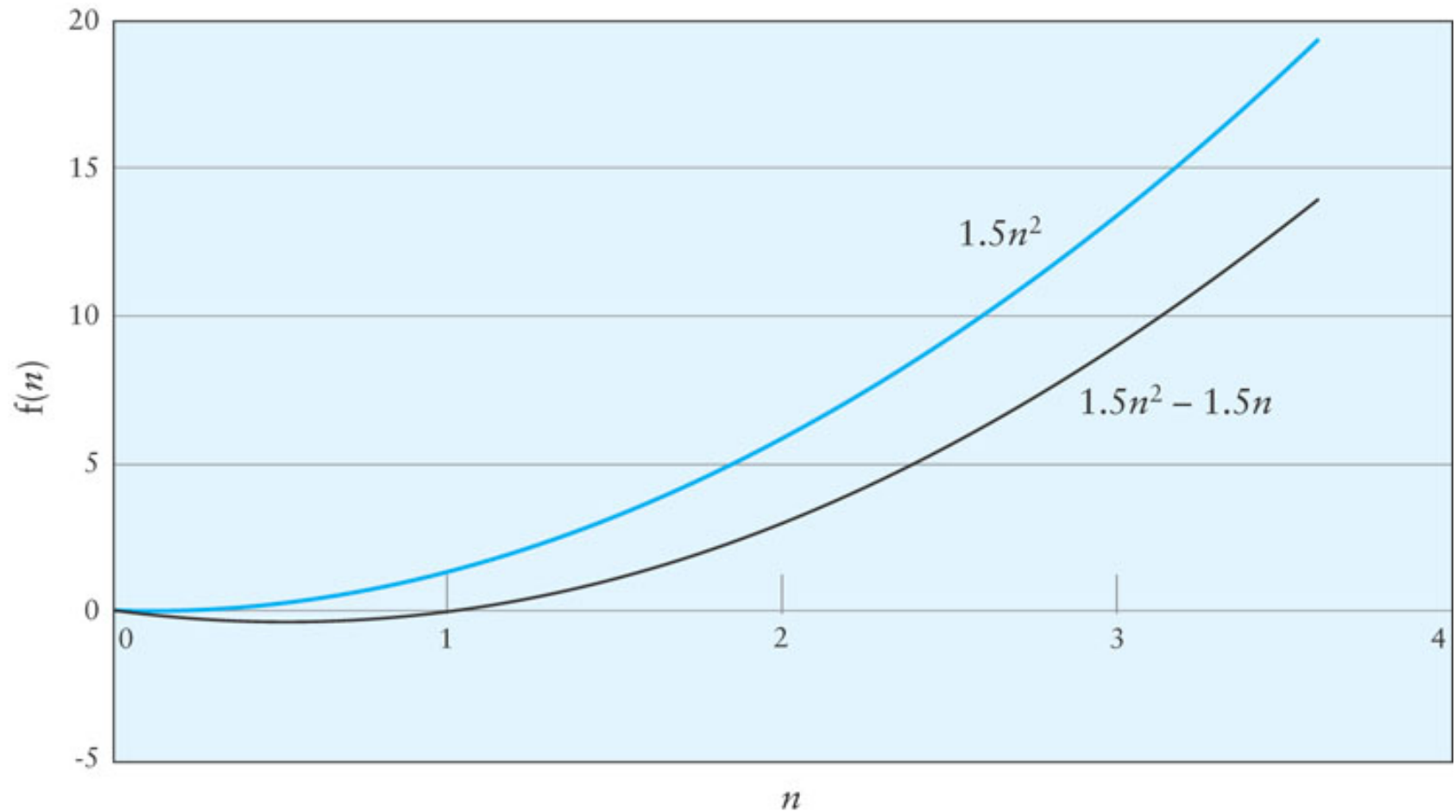
- Consider the following loop

```
for (int i = 0; i < n; i++) {  
    for (int j = i + 1; j < n; j++) {  
        3 simple statements  
    }  
}
```
- $T(n) = 3(n - 1) + 3(n - 2) + \dots + 3$
- Factoring out the 3,
 $3(n - 1 + n - 2 + n - 3 + \dots + 1)$
- $1 + 2 + \dots + n - 1 = (n \times (n-1))/2$

Big-O Example 2 (cont.)

- Therefore $T(n) = 1.5n^2 - 1.5n$
- For values of $n > 1$, $1.5n^2 > 1.5n^2 - 1.5n$
- Therefore $T(n)$ is $O(n^2)$ when n_0 is 1 and c is 1.5

Big-O Example 2 (cont.)



Symbols Used in Quantifying Performance

Symbol	Meaning
$T(n)$	The time that a method or program takes as a function of the number of inputs, n . We may not be able to measure or determine this exactly.
$f(n)$	Any function of n . Generally, $f(n)$ will represent a simpler function than $T(n)$, for example, n^2 rather than $1.5n^2 - 1.5n$.
$O(f(n))$	Order of magnitude. $O(f(n))$ is the set of functions that grow no faster than $f(n)$. We say that $T(n) = O(f(n))$ to indicate that the growth of $T(n)$ is bounded by the growth of $f(n)$.

A method for performing addition

- ```
int add (int a, int b){
 return a+b;
}
```

What is the time complexity for the above?

It takes constant time, so  $O(1)$

# Matrix Multiplication

- ```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        double sum = 0;  
        for (int k = 0; k < n; k++) {  
            sum += a[i][k] * b[k][j];  
        }  
        c[i][j] = sum;  
    }  
}
```
- What is the time complexity of the above?
- Since there are 3 nested for loops, the time complexity is $O(n^3)$

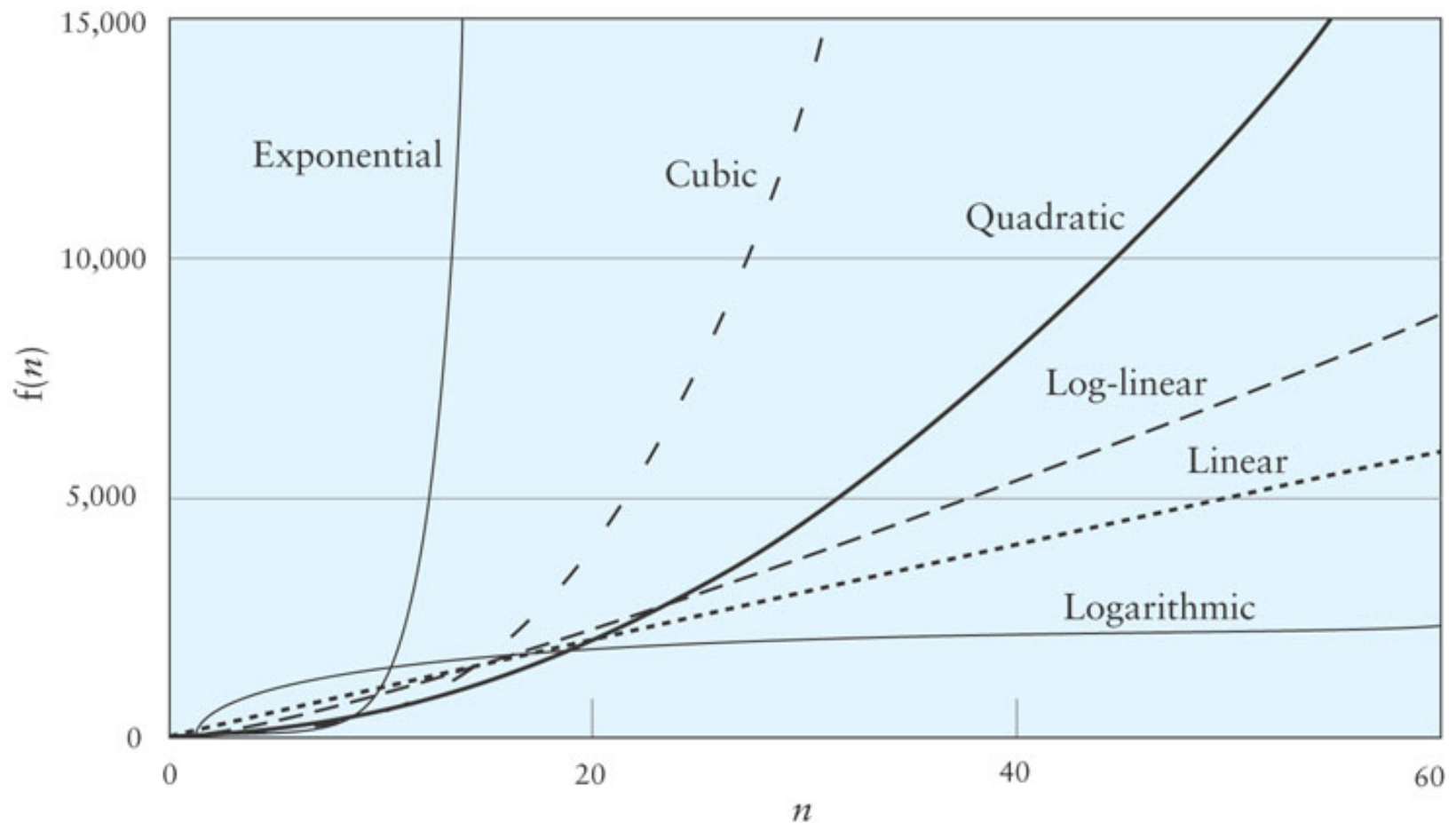
An interesting loop

- ```
for (int i = 1; i <= n; i *= 2) {
 for (int j = 0; j < n; j++) {
 count++;
 }
}
```
- What is the time complexity for the above?
- There are 2 loops:
  - The inner loop is  $O(n)$
  - The outer loop is  $O(\log n)$
  - Hence, the overall complexity is  $O(n \log n)$

# Common Growth Rates

| Big-O         | Name        |
|---------------|-------------|
| $O(1)$        | Constant    |
| $O(\log n)$   | Logarithmic |
| $O(n)$        | Linear      |
| $O(n \log n)$ | Log-linear  |
| $O(n^2)$      | Quadratic   |
| $O(n^3)$      | Cubic       |
| $O(2^n)$      | Exponential |
| $O(n!)$       | Factorial   |

# Different Growth Rates



# Effects of Different Growth Rates

| $O(f(n))$     | $f(50)$                | $f(100)$              | $f(100)/f(50)$         |
|---------------|------------------------|-----------------------|------------------------|
| $O(1)$        | 1                      | 1                     | 1                      |
| $O(\log n)$   | 5.64                   | 6.64                  | 1.18                   |
| $O(n)$        | 50                     | 100                   | 2                      |
| $O(n \log n)$ | 282                    | 664                   | 2.35                   |
| $O(n^2)$      | 2500                   | 10,000                | 4                      |
| $O(n^3)$      | 12,500                 | 100,000               | 8                      |
| $O(2^n)$      | $1.126 \times 10^{15}$ | $1.27 \times 10^{30}$ | $1.126 \times 10^{15}$ |
| $O(n!)$       | $3.0 \times 10^{64}$   | $9.3 \times 10^{157}$ | $3.1 \times 10^{93}$   |

# Is Big-O useful?

- Big-O notation is really only most useful for large  $n$ . The suppression of low-order terms and leading constants is misleading for small  $n$ . Be careful of functions such as
  - $n^2 + 1000n$
  - $0.0001n^5 + 10^6n^2$
  - $10^{-4}2^n + 10^5n^3$
- Large constants on the more significant terms signify *overhead*, for example when comparing
  - $8n \log n$  and  $0.01n^2$
  - note that the former is worse until  $n$  gets up to 10,710.

# Is Big-O useful?

- Big-O analysis must be considered the very first step in a progressive process of refining the analysis of an algorithm to reveal more details about its properties.
- **Big-O is only an upperbound; it is not a tight upperbound.**
- If an algorithm is in  $O(n^2)$  it is also in  $O(n^3)$ ,  $O(n^{100})$ , and  $O(2^n)$ . So it's not really right to say that all complexity functions in  $O(2^n)$  are terrible, because that set properly includes the set  $O(1)$ .

# Algorithms with Exponential and Factorial Growth Rates

- Algorithms with exponential and factorial growth rates have an effective practical limit on the size of the problem they can be used to solve
- With an  $O(2^n)$  algorithm, if 100 inputs takes an hour then,
  - 101 inputs will take 2 hours
  - 105 inputs will take 32 hours
  - 114 inputs will take 16,384 hours (almost 2 years!)

# Algorithms with Exponential and Factorial Growth Rates (cont.)

- Encryption algorithms take advantage of this characteristic
- Some cryptographic algorithms can be broken in  $O(2^n)$  time, where  $n$  is the number of bits in the key
- A key length of 40 is considered breakable by a modern computer, but a key length of 100 bits will take a billion-billion ( $10^{18}$ ) times longer than a key length of 40

# Lower Bounds

- Big-O describes the upper bound, for the worst case scenario
- Similarly, Big-Omega describes the lower bound, for the best case scenario
  - What is the least amount of resource an algorithm needs for some class of input

For  $\mathbf{T}(n)$  a non-negatively valued function,  $\mathbf{T}(n)$  is in set  $\Omega(g(n))$  if there exist two positive constants  $c$  and  $n_0$  such that  $\mathbf{T}(n) \geq cg(n)$  for all  $n > n_0$ .<sup>1</sup>

---

**Example 3.7** Assume  $\mathbf{T}(n) = c_1n^2 + c_2n$  for  $c_1$  and  $c_2 > 0$ . Then,

$$c_1n^2 + c_2n \geq c_1n^2$$

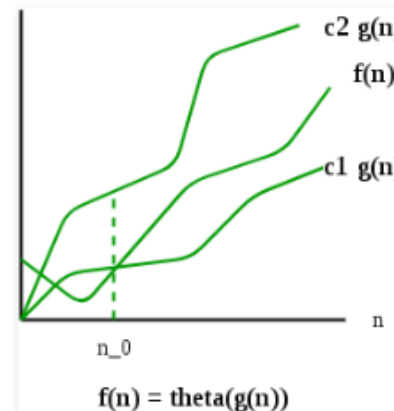
for all  $n > 1$ . So,  $\mathbf{T}(n) \geq cn^2$  for  $c = c_1$  and  $n_0 = 1$ . Therefore,  $\mathbf{T}(n)$  is in  $\Omega(n^2)$  by the definition.

---

# Θ Notation (Big-Theta)

- The definitions for big-Oh and  $\Omega$  give us ways to describe the upper bound for an algorithm (if we can find an equation for the maximum cost of a particular class of inputs of size  $n$ ) and the lower bound for an algorithm (if we can find an equation for the minimum cost for a particular class of inputs of size  $n$ ). When the upper and lower bounds are the same within a constant factor, we indicate this by using  $\Theta$  (big-Theta) notation.
- An algorithm is said to be  $\Theta(h(n))$  if it is in  $O(h(n))$  and it is in  $\Omega(h(n))$ .
- $f(n) = \Theta(g(n))$  if there exist positive constants  $c_1$ ,  $c_2$  and  $n_0$  such that,  
 $c_1 * g(n) \leq f(n) \leq c_2 * g(n)$  for all  $n \geq n_0$

$$3n^3 + 6n^2 + 6000 = \Theta(n^3)$$



- Questions..??